

Reliability audit

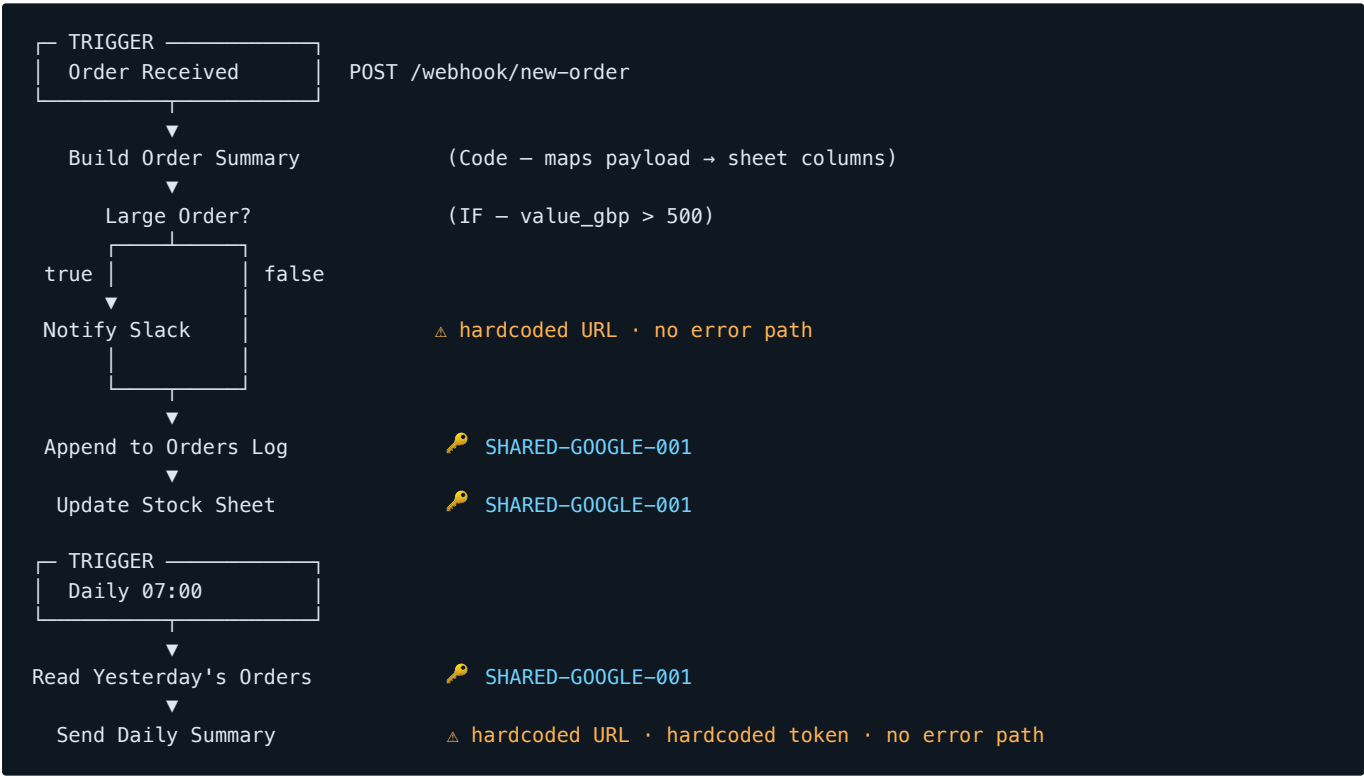
Subject: Shop Orders → Slack, Sheet & Daily Summary · 9 nodes, 2 triggers

AUDITED	METHOD	AUDITOR	IMPLEMENTATION
31 July 2026	Exported JSON only — no live access	Ben Foster	None — review only

This is a demonstration. The workflow audited here is one I wrote myself, as an example of how this kind of integration usually looks when it has been built quickly by someone competent. It is not a client's workflow and no client data appears anywhere in it. It is deliberately ordinary — it works, and if you ran it today it would do what it is supposed to.

The workflow does what it was built to do, and there are five ways it can fail without anyone finding out — three of which lose order data outright. Estimated **20 hours** to close everything, of which the first **two hours** remove the largest single risk.

1 Dependency map



Two independent flows share one credential. SHARED-GOOGL-001 authenticates three nodes across both flows. It is the only thing joining them, and nothing in the workflow records that. Revoke or rotate it for any reason and the order log, the stock sheet and the daily summary all stop — three failures, two flows, one cause, and no error that names it. Four external services, none monitored.

2 Failure-risk register

Every point where this workflow can fail *without telling anyone*, classified by consequence.

#	FAILURE	CLASS	WHAT ACTUALLY HAPPENS	FIX
R1	No idempotency anywhere between the webhook and the writes	DUPLICATES	The shop retries the webhook — every shop platform does, on timeout or 5xx. The same order posts twice, appends a second row to the orders log, and decrements stock a second time . Stock drift has no audit trail and is found at stocktake.	4h
R2	Notify Slack runs <i>before</i> the sheet write and has no error output	LOSES DATA	Slack has a bad thirty seconds → the node throws → the execution stops → Append to Orders Log never runs . The order is never recorded. This affects only orders over £500 , because small orders bypass Slack entirely — so the orders you lose are the largest ones, and the pattern looks random.	2h
R3	Build Order Summary reads line_items[0] only	LOSES DATA	Any multi-line order is silently truncated to its first line. Verified: a 3-line order produced 1 row and dropped 6 units . The order total stays correct, so the sheet looks internally consistent and the loss is invisible without reconciling against the shop.	3h
R4	No guard on the payload shape	STOPS SILENTLY	An order with an empty line_items array throws TypeError — verified . The execution fails, nothing alerts, and the order is lost. Refunds, zero-value and subscription-renewal payloads are the usual triggers.	2h
R5	Nothing observes either flow	STOPS SILENTLY	If the shop stops calling the webhook, or the schedule is deactivated by an edit, or the token expires — there is no error, no alert, no evidence. The first signal is a human noticing the sheet stopped growing, which on a quiet week takes a fortnight.	3h

R2 is the one to fix first. Two hours, and it is the only finding here that loses your highest-value orders specifically.

3 The five standing checks

Run on every audit, so that the absence of a finding is as meaningful as its presence.

ABSENT	Duplicate suppression No fingerprint, no key, no lookup. Nothing between Order Received and either write distinguishes a retry from a new order.
ABSENT	Dead-letter capture No node sets retryOnFail or onError. A failure is an aborted execution — the payload is not preserved anywhere it can be replayed from.
NOT APPLICABLE	Unbounded state growth The workflow keeps no state at all. That is <i>why</i> the first check fails — it is not a clean bill of health, it is the same finding seen from the other side.
2 IN PLAIN TEXT	Credential handling & hardcoded secrets A webhook URL in Notify Slack, and a bearer token in the Authorization header of Send Daily Summary. Both sit in node parameters, so both appear in every export and every execution log . Separately, one credential spans three nodes across two independent flows.
NO	Would anything notice silence? No canary, no heartbeat, no downstream freshness check. Both flows can stop permanently with no signal.

4 Prioritised backlog

Ordered by risk against effort, estimated in hours. Written so you can hand it to any developer — including one who isn't me.

#	TASK	HOURS	CLOSES	WHY HERE
1	Write to the orders sheet before notifying Slack , and give <code>Notify Slack</code> an error output that continues rather than aborting	2	R2	Highest risk, lowest effort. A notification failing should never prevent the record. A wiring change, not new code.
2	Iterate every line item — emit one row per line, not per order	3	R3	Silent data loss on exactly the orders that matter most; multi-line means larger baskets.
3	Add an idempotency key on <code>order_id</code> — look up before writing	4	R1	Highest severity, but genuinely more work: needs a store, and a decision on what "already seen" means for legitimate re-orders.
4	Format dates from local parts , not <code>toISOString()</code>	1	—	An order placed at 00:30 on 1 July is recorded as 30 June. Verified . One hour, and it silently misfiles orders across month and quarter boundaries.
5	Validate the payload before mapping — reject and log anything without usable line items	2	R4	Turns a crash into a handled, visible rejection.
6	Move both hardcoded secrets into the credential store	2	—	The bearer token is in plain text in every export you have ever sent anyone, including this audit.
7	Add a daily canary — alert if no row written in 26 hours	3	R5	26 not 24, so a quiet Sunday or a late run doesn't cry wolf.
8	Write a one-page runbook covering the gaps in §5	3	—	Last by risk, first by what it costs you the day someone is off sick.

Total: 20 hours. Items 1–2 are five hours and remove both data-loss findings.

5 Runbook gap list

What a competent person covering for you **could not work out** from the workflow alone.

1. **Which Google account is SHARED-GOOGLE-001**, who controls it, and what else depends on it.
2. **Which channel the webhook posts to**. The URL is opaque by design — you cannot tell from it, and there is no note anywhere.
3. **Which spreadsheets the two document IDs refer to**. Two raw IDs, no names, no links.
4. **What to do when the daily summary doesn't arrive**. No documented first check, and the likeliest causes — expired token, deactivated schedule, mail API rejection — look identical from the outside: nothing happens.
5. **Whether a failed execution is safe to re-run**. It is **not** — with no idempotency, a manual retry duplicates the order row and double-counts stock. Nothing says so, and re-running a failed execution is the first thing anyone tries.
6. **Where the mail API bearer token came from**, whose account it bills, and how to rotate it.

6 What was not examined

- **No implementation**. Nothing in the backlog has been built.
- **No live access**. Findings come from the exported JSON. Anything that only appears at runtime — real rate limits, real quota behaviour, the shop's true retry policy — is out of scope and flagged as an assumption rather than a fact.
- **No security or compliance opinion**. The hardcoded-secret finding is a reliability and handover finding, not a penetration test and not a certification statement.

- **No verdict on whether the workflow is "good".** It was built to do a job and it does it. This is a fixed examination, not a judgement.

Findings R3, R4 and backlog item 4 were verified by **executing** the Build Order Summary node's actual source against sample payloads, not by reading it. The remaining findings are structural and visible in the exported JSON: no node in this workflow sets `retryOnFail` or `onError`, and `SHARED-GOOGLE-001` appears on three of the nine nodes. The verifier is public and runs with one command, no API key: node `04-audit-example/verify-findings.mjs`